

Online Synthesis for Runtime Enforcement of Safety in Multiagent Systems

Dhananjay Raju , Sudarshanan Bharadwaj , Member, IEEE, Franck Djeumou ,
and Ufuk Topcu , Member, IEEE

Abstract—We study the problem of enforcing safety in multiagent systems at runtime by modifying the system behavior if a potential safety violation is detected. Traditional runtime enforcement methods that solve a reactive synthesis problem at design time have two significant drawbacks. First, these techniques do not scale as one has to take into account all possible behaviors from every agent, and this is computationally prohibitive. Second, these approaches require every agent to know the state of every other agent. We address these limitations through a new approach where online modifications to behavior are synthesized onboard every agent. There is an *enforcer* onboard every agent, which can modify the behavior of only the corresponding agent. In this approach, which is naturally decentralized, the enforcer on every agent has two components: 1) a pathfinder that corrects the behavior of the agent and 2) an ordering mechanism that dynamically modifies the priority of the agent. The current priority of an agent determines if the enforcer uses the pathfinder to modify the behavior of the agent. We derive an upper bound on the maximum deviation for any agent from its original behavior; that is, all agents make progress. We prove that the worst case synthesis time is quadratic in the number of agents at runtime as opposed to exponential at design time for the existing methods that rely on design-time computation merely. Additionally, we prove the completeness of the technique under some mild assumptions; that is, if the agents can progress safely, then enforcers will find this behavior. We test the technique in collision avoidance scenarios. For 50 agents in a 50×50 grid world modeling the common workspace for the agents, the online synthesis requires only a few seconds per agent whenever a potential collision is detected. In contrast, the centralized design-time synthesis of shields for a similar setting is intractable beyond four agents in a 5×5 grid world.

Index Terms—Multiagent systems, runtime enforcement, safety, synthesis.

I. INTRODUCTION

ENSURING the safety of multiagent systems is a crucial and challenging problem. We study this problem in a setting

Manuscript received January 8, 2020; revised October 13, 2020, October 13, 2020, and October 14, 2020; accepted February 9, 2021. Date of publication February 24, 2021; date of current version August 24, 2021. This work was supported in part by AFRL under Grant FA9550-19-1-0169, in part by AFOSR under Grant FA9550-19-1-0005, and in part by DARPA under Grant D19AP00004. Recommended by Associate Editor T. Wettergren. (Corresponding author: Dhananjay Raju.)

The authors are with the University of Texas at Austin, Austin, TX 78712 USA (e-mail: draju@cs.utexas.edu; suda.b@utexas.edu; fdjeumou@utexas.edu; utopcu@utexas.edu).

Digital Object Identifier 10.1109/TCNS.2021.3061900

where: 1) the agents do not know the state of the other agents; 2) the agents can only communicate if they are in a communication group, which depends on spatial proximity; and 3) each agent can share only a limited amount of information with the other agents.

Runtime enforcement is one approach for ensuring safety for multiagent systems [1]. Enforcers typically monitor the behavior of the system and modify it if they detect anything potentially unsafe. Shielding is an approach to runtime enforcement [2], [3]. A shield is typically assumed to be aware of and is able to affect all the agents in the system instantaneously [4]. Thus, shields require global information about the state of the system. However, global information on the state of all the agents is often difficult to obtain in multiagent systems. There has been some work in relaxing these assumptions using *localized shields* that have awareness and authority over only the agents in their local region. However, no genuinely *decentralized* approach, in which a shield onboard each agent can modify only the corresponding agent's behavior, exists [5]. In such an approach, there would be no entity that has global information on the state space of the entire system.

Without global information of the state, guaranteeing safety is, in general, undecidable [6]. Thus, we focus solely on enforcing *local* safety properties, which is a subset of general temporal safety properties. A safety property is local if it can be enforced in the entire multiagent system by enforcing it within each communication group. Essentially, shields are partial functions from the current states of the agents to the next states. Existing methods find this partial function by solving a reactive synthesis problem at design time [2], [4], [5]. However, it is computationally prohibitive in the case of multiagent systems since the resulting safety game has to take into account all possible behaviors from every agent [5].

We formulate the synthesis of modified safe behavior of an agent as a graph search problem. More specifically, we assume that agents know the intended behaviors of the other agents in its communication group and, hence, an onboard enforcer can modify an agent's behavior, taking into account the behavior of the other agents in the same group. If the system continues to remain unsafe after the agent has changed its behavior, then the other agents are forced to change their behaviors. Thus, the synthesis of safe behavior for all agents in a communication group can be framed as a sequence of graph searches. This technique is similar to hierarchical path planning [7]. In particular, we synthesize safe behavior online when required,

i.e., when the intended trajectories of the agents violate a safety requirement. However, such an online approach to synthesizing new behaviors may create scenarios where some agents may never progress. That is, the behaviors of some agents may be perpetually modified to ensure safety.

In this article, we present a novel *decentralized* framework for *online* synthesis for runtime enforcement. The enforcer onboard each agent issues modifications to the behavior of its corresponding agent in an order according to their *priority* using graph search. The framework uses a novel decentralized *ordering mechanism* to dynamically maintain the agent's priorities to ensure that every agent can make progress according to its intended behaviors. We assume that the agents have agreed on this mechanism. In addition, it is possible to compute the order between any two agents (the total order relation corresponding to the priorities) on the fly using only the flags that are local to the two agents. Moreover, only the corresponding agents can modify these flags. The presented ordering mechanism provably guarantees that every agent can acquire the highest priority in a finite length of time; hence, livelocks are avoided. The online synthesis approach performs local behavior modification as needed; this circumvents the state-space explosion.

The proposed approach is similar to *cooperative path planning* in multiagent systems, which is a PSPACE-hard problem [8]. Hierarchical cooperative A* (HCA*) is a decentralized approach that uses fixed priorities on agents and makes a plan for an agent while respecting the plans of the agents with higher priorities [7]. However, HCA* may require the agents to change their plan continuously and, therefore, cannot guarantee finite-time progress [7]. Proposed approaches that achieve completeness and produce optimal paths [9], [10] are either nontractable or rely on global information. The method in [11] relaxes the reliance on global information; however, it still falls back to using it as a last resort. In addition, if the agents in the system can idle, we provide a condition that guarantees *completeness*. That is, if there exists a safe behavior, then the enforcer can guarantee safety. This level of decentralization while ensuring system-level safety is possible because the decentralized priority exchange mechanism we present ensures the absence of livelocks. In the existing techniques, livelocks have to be detected, which requires global information.

We evaluate the performance of the decentralized enforcer framework in the specific context of collision avoidance for multiagent systems. Collision avoidance is a well-studied topic. In [12], an online reinforcement learning method is used for path planning. In [13], a discriminative classifier, which is used to map gas distribution, is trained online. An entropy-based artificial potential field or entropy-based particle swarm optimization algorithm is used in conjunction with the classifier for path planning. In contrast with the previous techniques, the method in this article modifies the trajectories of the agents only when collisions are detected. Additionally, the method guarantees progress, i.e., no single agent keeps deviating continuously to keep the system collision free.

In [14], a distributed optimal control method based on the generalized reduced gradient is used for path planning. However, the method provides no guarantees for completeness. In contrast, we ensure a path exists if a centralized enforcer (referred to as

a shield in [5]) is able to find a path. More specifically, under certain assumptions, the technique in this article will synthesize decentralized enforcers that guarantee safety for all agents if there is a centralized enforcer that can guarantee safety. In contrast to the work in [15], the environment in this article is a discrete graph, and the agents take one unit of time to move from one location to the next location, i.e., the agents have uniform dynamics that do not depend on their location. These restrictions enable progress and completeness.

Contributions

To our best knowledge, this article presents the first approach where the enforcement of safety properties is viewed through the lens of cooperative path planning. The existing formulation for runtime enforcement through shielding uses reactive systems. However, this is unsuitable and cumbersome for the online approach. Therefore, we provide a new formulation where the enforcers are tuples of partial functions. An extra benefit of such an approach is that the *joint* behavior of all the agents can be directly expressed as a functional composition. Finally, we prove the resulting enforcers also satisfy the following properties [16].

- 1) *Correctness*: The modified system behavior satisfies all the safety properties.
- 2) *Minimal Deviation*: If there is no safety violation among the agents in a communication group, then the enforcers on the agents in the group must not modify the behavior of the agents.
- 3) *Boundedness*: The deviation from the original behavior must be finite. We additionally show that the maximum deviation is linear in the number of agents.
- 4) *Completeness*: If a centralized stabilizing shield can guarantee correctness, the enforcers will also guarantee correctness.

By construction, the enforcers do not require global information. Additionally, we prove that the worst case synthesis time for each agent is at most quadratic in the number of agents.

II. PRELIMINARIES

Following the notations in [17]–[19], $\mathbb{B} = \{\top, \perp\}$ is the domain of Booleans.

A finite (infinite) word over a set Σ of elements is a finite (infinite) sequence $w = a_0 a_1 \dots a_n$ of elements of Σ . The length of w is $|w|$. ϵ_Σ denotes the empty word over Σ or ϵ when the context is clear. $w[p : q]$ denotes the subword $a_p a_{p+1} \dots a_q$. The concatenation of two words w and w' is denoted as $w \cdot w'$. A word w' is a prefix of a word w , denoted as $w' \leq w$, whenever there exists a word w'' such that $w = w' \cdot w''$, and $w' < w$ if additionally $w' \neq w$. w is said to be an extension of w' . The sets of all words and all nonempty words are denoted by Σ^* and Σ^+ , respectively. $\Sigma^{\leq k}$ denotes all words of length at most k . 2^A denotes the power set of the set A . A language or a property over Σ is any subset L of Σ^* . If $a, b \in \mathbb{N}_{\geq 0}$ and $a < b$, then $[a, b]$ denotes the set $\{a, a+1, a+2, \dots, b-1, b\}$.

Let $G = (V, E)$ be a directed graph, where V is a finite set of nodes, and E is a finite set of edges. The *distance* $d(u, v)$ between two vertices u and v is defined as the length of a shortest directed path from u to v . Let $\mathcal{U}$ denote a set of node labels and

Σ denote a set of edge labels. $\mathbb{T} = \{0, 1, 2, \dots, \infty\}$ is a discrete set of time indices. A graph with node labels and edge labels is called a *labeled graph* [20]. An *edge labeling* is a function $\mathcal{Y} : E \times \mathbb{T} \rightarrow \Sigma$. A *node labeling* is a function $\mathcal{X} : V \times \mathbb{T} \rightarrow 2^{\mathcal{U}}$. A node labeling $\mathcal{X}$ is consistent at time t if $\mathcal{X}$ partitions V , i.e., if for any u and v , $\mathcal{X}(v, t) \cap \mathcal{X}(u, t) \neq \emptyset$ implies $u = v$.

An *environment* is a tuple $(G, \mathcal{Y})$ where G is a labeled graph and $\mathcal{Y}$ is an edge labeling. $\mathcal{Y}(e, t)$ is the label of edge e at time t . The environment is said to be *static* if the associated edge labeling is time invariant. For a static environment $(G, \mathcal{Y})$, $\delta : V \times \Sigma \rightarrow 2^V$ is called the *transition function*. δ is *deterministic*, if for any $v, v_1, v_2 \in V$ and $s \in \Sigma$, $v_1, v_2 \in \delta(v, s)$ implies $v_1 = v_2$. The extended transition function $\hat{\delta} : V \times \Sigma^* \rightarrow 2^V$ is defined recursively as $\hat{\delta}(v, \epsilon) = v$ and $\hat{\delta}(v, w \cdot a) = \delta(\hat{\delta}(v, w), a)$. A static environment is deterministic, if the associated transition function δ is deterministic. A word $w = w_0 w_1 w_2 \dots$ is said to *induce a path* in a graph G starting at vertex v_0 if there exists a sequence of vertices $v_1 v_2 v_3 \dots$ such that $v_i \in \delta(v_{i-1}, w_{i-1})$. In a static deterministic environment, the *final state* induced by a finite word $w = w_0 w_1 \dots w_n$ starting at v is the node $\hat{\delta}(v, w)$.

A *trajectory* p in a static environment $(G, \mathcal{Y})$ is a pair (v, w) , where $w \in \Sigma^*$ is a finite word such that w induces a path in G starting at vertex v . The final state of a trajectory $p = (v, w)$ is the final state induced by w on v and is given by $\hat{\delta}(v, w)$. The concatenation of a trajectory p and a word w' is $p \cdot w' = (v, w \cdot w')$. For trajectory p , we denote its *subtrajectory* $(v, w[i : \ell])$ by $p[i : \ell]$. A *joint trajectory* is a finite set of trajectories.

For any vertex label u , $p = (v, w)$ is a trajectory for u at time t if $u \in \mathcal{X}(v, t)$ and w induces a path from v . Define the final state of u through p as the final state of (v, w) . The final state of u through trajectory $p = (v, w)$ is $(\hat{\delta}(v, w))$ and is denoted as $u \xrightarrow{w}_v cr \rightarrow \hat{\delta}(v, w)$. Let $\mathcal{X}(v, t) = \{u\} \cup X$, (v, w) be a trajectory and $\mathcal{X}(\hat{\delta}(v, w), t) = Y$. If agent u follows trajectory (v, w) , then at time $t + |w|$, the vertex label of v is X and the vertex label of $\hat{\delta}(v, w)$ is $Y \cup \{u\}$.

Given an n -tuple of symbols $e = (e_1, \dots, e_n)$, for $i \in [1, n]$, $\prod_i e$ is the projection of e on its i th element denoted as $(\prod_i e \stackrel{\text{def}}{=} e_i)$. $\mathcal{R}_i(e, x)$ replaces the i th element of e with x , i.e., $\mathcal{R}_i(e, x) = (e_1, \dots, e_{i-1}, x, e_{i+1}, \dots, e_n)$.

III. ONLINE ENFORCERS

Environment and Agents: We model the region of operation of the agents as a deterministic environment $(G, \mathcal{Y})$ with a consistent node labeling $\mathcal{X}$. The set of all agents is $\mathcal{U}$, and it is the same as the set of all node labels. At time $t \in \mathcal{T}$, an agent u is said to be at location v if $u \in \mathcal{X}(v, t)$. We note that a location v can have more than one label, i.e., more than one agent can be at the same location at a given time. The set Σ of edge labels of G is the set of actions available to the agents. An agent can move from a vertex v to a vertex v' in one time unit through an action $s \in \Sigma$, if and only if there is an edge with the corresponding edge label s between v and v' . If (v, w) is a trajectory, and agent u follows the trajectory starting at time t , then at time $t + i$, where i is an integer such that $i \leq |w|$, the state of the agent u is $\hat{\delta}(v, w[0 : i])$. Furthermore, $\mathcal{X}(v, t) = \{u\}$

and $\mathcal{X}(\hat{\delta}(v, w[0 : i]), t + i) = \emptyset$. For any trajectory (v, w) , we drop the initial vertex when it is clear. At time $t + i$, $\mathcal{X}(v, t) = \emptyset$ and $\mathcal{X}(\hat{\delta}(v, w[0 : i]), t + i) = \{u\}$. Boolean goal $_{u,t}$ is true when agent u reaches its final state at time t , which is referred to as the agent having *completed the goal*. Formally

$$\text{goal}_{u,t} = \begin{cases} \top, & \text{if } u \text{ has reached its final state} \\ & \text{following the trajectory } (v, w) \\ \perp, & \text{otherwise.} \end{cases} \quad (1)$$

Associated with any agent is a unique priority from $[1, |\mathcal{U}|]$, defined as $\text{priority} : \mathcal{U} \times \mathbb{T} \rightarrow [1, |\mathcal{U}|]$, such that $\text{priority}(u_1, t) = \text{priority}(u_2, t)$ implies $u_1 = u_2$. At any time t , the priorities of the agents induce a total order $\prec_t$ among them. For agents u_1 and u_2 in $\mathcal{U}$, $u_1 \prec_t u_2$ if and only if $\text{priority}(u_1, t) < \text{priority}(u_2, t)$.

Communication: The agents in the system can communicate when they are close to each other. Moreover, two agents u_i and u_j can also communicate if there is a sequence of agents $c_1 \dots c_k$ such that c_1 is u_i , and c_k is u_j and there is a path of length less than or equal to d between agents c_i and c_{i+1} . Here, d is a positive integer referred to as the *communication constant*. At any time t , $\mathcal{U}_{u_i}(t)$ denotes the communication group of agent u_i . The agents in the same communication group know the partial trajectories of the other agents in the group up to length ℓ . Formally, every agent u knows the partial trajectory $(v_{u'}, w_{u'})[0 : \ell]$ of every other agent u' in its communication group, where $(v_{u'}, w_{u'})$ is the trajectory for agent u' . Henceforth, ℓ is referred to as the *look-ahead*.

Safety Functions: A *safety* property φ is a function $\varphi : 2^P \rightarrow \mathbb{B}$ from the vertex labeling to Booleans, where $P = V \times 2^{\mathcal{U}}$. As the vertex labels themselves depend on time, we extend the notion of safety to safety at time t . If the vertex labeling at time t is safe, then $\varphi(t) = \top$.

Next, we extend the notion of safety to trajectories. For every agent u , denote its trajectory by (v_u, w_u) . The system is *safe on the trajectories*, if the system is safe at all times. Formally, $\varphi(\mathcal{U}) = \top$ if for all $t \in [0, m]$, $\varphi(t) = \top$, where $m = \max\{|w_u| : u \in \mathcal{U}\}$.

Recall the agents in the system have only limited communication. Therefore, we are interested in a subclass of safety functions that can be enforced across the system by enforcing them locally in every communication group. Suppose φ_i is a safety property such that $\varphi_i(t) = \top$ if and only if the agents in the communication group $\mathcal{U}_i$ are safe. Then, the property $\varphi(t)$ defined as

$$\varphi(t) \stackrel{\text{def}}{=} \bigwedge_{i \in 1 \dots |\mathcal{U}|} \varphi_i(t) \quad (2)$$

is a *local* safety property. Observe that the safety function defined in Scenario 3 is a local safety property.

Enforcers: Informally, the purpose of an enforcer is to take a (possibly incorrect) trajectory produced by a running system and to transform it into a trajectory that is safe with respect to a local safety function φ that we want to enforce. Abstractly, an enforcer can be seen as a function that transforms trajectories.

Denote by $S[u, t]$ the enforcer acting on agent u at time t . $S[u, t]$ is a pair of *partial* functions $\langle S_1[u, t], S_2[u, t] \rangle$. $S_1[u, t]$

accepts a finite trajectory for each agent in the system and returns a modified trajectory for agent u . $S_2[u, t]$ accepts a vector of current priorities and a vector of Booleans $\text{goal}_{u,t}$ for the agents in the system. It returns a vector of priorities with only the priority of its corresponding agent u possibly changed. Formally, the enforcer on agent u at time t is a pair of partial functions $\langle S_1[u, t], S_2[u, t] \rangle$ such that

$$S_1[u, t] : \mathcal{L}_1^{|U|} \rightarrow \mathcal{L}_1^{|U|} \text{ and} \quad (3)$$

$$S_2[u, t] : [1, |U|]^{|U|} \times \mathbb{B}^{|U|} \rightarrow [1, |U|]^{|U|} \times \mathbb{B}^{|U|} \quad (4)$$

where $\mathcal{L}_1 = [1, |U|] \times V \times \Sigma^*$. The previous definition of an enforcer is quite general as both the input trajectory and the modified trajectory can be of the arbitrary length. Next, we introduce (ℓ, ℓ') -enforcers. For every agent in the system, these enforcers accept trajectories of length at most ℓ and return a new trajectory of length at most $\ell' \geq \ell$. Formally, an (ℓ, ℓ') -enforcer on agent u at time t is a pair of partial functions $\langle S_1[u, t], S_2[u, t] \rangle$ such that

$$S_1[u, t] : \mathcal{L}_I^{|U|} \rightarrow \mathcal{L}_O^{|U|} \text{ and} \quad (5)$$

$$S_2[u, t] : [1, |U|]^{|U|} \times \mathbb{B}^{|U|} \rightarrow [1, |U|]^{|U|} \times \mathbb{B}^{|U|} \quad (6)$$

where $\mathcal{L}_I = [1, |U|] \times V \times \Sigma^{\leq \ell}$ and $\mathcal{L}_O = [1, |U|] \times V \times \Sigma^{\leq \ell'}$.

Composition of Enforcers: When multiple agents act in the same system, their trajectories and their priorities are modified only by their respective enforcers. However, the individual enforcers act together to make the system safe. The joint behavior of enforcers are captured by functional composition. We first define *composition of enforcers* for two agents u_1 and u_2 at time t . If $u_1 \prec_t u_2$, then

$$\begin{aligned} S[u_1, t] \circ S[u_2, t] \\ = S[u_2, t] \circ S[u_1, t] \end{aligned} \quad (7)$$

$$= \langle S_1[u_2, t] \circ S_1[u_1, t], S_2[u_2, t] \circ S_2[u_1, t] \rangle. \quad (8)$$

This composition can be extended to an arbitrary number of enforcers by composing their constituent functions in the order $\prec_t$.

Properties of Enforcers: We define the desired properties for a set of enforcers. For any agent u in U , let $S[u, t]$ be the enforcer acting on agent u at time t , p_u its original trajectory at time t , and p'_u its modified trajectory at time t .

Definition 1 (Correctness): The enforcers $\{S[u, t] | u \in U\}$ are *correct* if the modified trajectories $\{p'_u | u \in U\}$ are safe and the final states are unchanged.

Definition 2 (Minimal deviation): An enforcer $S[u, t]$ is said to *cause minimum deviation* if for all $a \in \mathcal{U}_u(t)$, $\varphi_a(t) = \top$, then $p'_u = p_u$.

Definition 3 (Boundedness): The enforcers $\{S[u, t] | u \in U\}$ are *bounded*, if there exists ℓ and ℓ' in $\mathbb{N}$ such that all the enforcers are (ℓ, ℓ') -enforcers.

Definition 4 (Completeness): If there exists a centralized shield [16] that is correct, then the decentralized enforcers are correct.

We later prove that boundedness and correctness ensure that all agents progress in finite time while still guaranteeing the safety of the system. We now state the problem studied in this article.

Problem 1: Given a set $U = \{u_1, \dots, u_n\}$ of agents and a set $\{(v_u, w_u) | u \in U \text{ and } |w_u| \leq \ell\}$ of their trajectories, construct a set $\{S[u, t] | u \in U \text{ and } t \in \mathbb{T}\}$ of enforcers such that these enforcers are correct and cause minimum deviation, complete, and bounded.

In the following scenarios, we explain the concepts discussed in this section.

Scenario 1: In Fig. 1(a), blue and green agents operate in a grid world. The vertices of the underlying labeled graph G are the cells in the grid. There is an edge from a vertex to another, if they are adjacent in the grid (no diagonal edges). The set $\Sigma \stackrel{\text{def}}{=} \{l, r, t, d\}$ of edge labels is the set of actions corresponding to moving left, right, top, and down, respectively. The set $U = \{\text{blue}, \text{green}\}$ of vertex labels corresponds to the set of agents operating in the system. $\mathcal{X}((2, 4), 0) = \{\text{green}\}$ and $\mathcal{X}((4, 2), 0) = \{\text{blue}\}$, that is, blue agent is at (4,2) and green agent is at (2,4). The trajectory of the blue agent is $((4, 2), ll)$ and that of the green agent is $((2, 4), ddd)$. At time $t = 1$, the labeling function is $\mathcal{X}((3, 2), 1) = \{\text{green}\}$, $\mathcal{X}((2, 3), 1) = \{\text{blue}\}$, $\mathcal{X}((2, 4), 1) = \emptyset$, $\mathcal{X}((4, 2), 1) = \emptyset$. The blue and the green agents have reached their goals at $t = 3$. Therefore, $\text{goal}_{\text{blue}, 3} = \top$ and $\text{goal}_{\text{blue}, 2} = \text{goal}_{\text{blue}, 1} = \perp$. The final state of the blue agent is (1,2) and the green agent is (2,1). That is

$$\text{blue} \xrightarrow[(1,2)]{ll} \text{ and } \text{green} \xrightarrow[(2,4)]{ddd} (2,1).$$

Scenario 2: In Fig. 1(b), the trajectories of 5 agents are shown. The communication constant $d = 2$ and $\ell = 3$. At time 0, the $\mathcal{U}_{\text{blue}}(0) = \mathcal{U}_{\text{green}}(0) = \mathcal{U}_{\text{red}}(0) = \{\text{blue}, \text{green}, \text{red}\}$ and $\mathcal{U}_{\text{purple}}(0) = \mathcal{U}_{\text{black}}(0) = \{\text{purple}, \text{black}\}$. All agents in the first group know that the trajectory of the red agent is $((2, 5), ddd)$, the blue agent is $((2, 7), tt)$ and the green agent is $((4, 7), ll)$. In the second group, all agents know that the trajectory of the purple agent is $((6, 3), ll)$ and the black agent is $((6, 1), rr)$.

Scenario 3: Consider a safety function $\varphi : \mathbb{T} \rightarrow \mathbb{B}$ defined as

$$\varphi(t) = \begin{cases} \top & \text{if for any } v \in V | \mathcal{X}(v, t) | = 1 \\ & \text{and } \mathcal{X}(u, t) = \mathcal{X}(v, t) \text{ implies } u = v \\ \perp & \text{otherwise.} \end{cases} \quad (9)$$

Simply if X is the vertex labeling at some time, X is *safe* if there is only one agent at any location at any given time and any agent can be present at only one location at any time (consistent label). In the grid world in Fig. 1(c), the system is safe at all times. In the grid world in Fig. 1(b), the system is unsafe at time $t = 2$ as the blue and the green agents occupy the same location.

Scenario 4: Consider safety as described in Scenario 3. Then, in Fig. 1(c), the system is safe on trajectories $ltllld$ and ddd for the blue and green agents, respectively. However, in Fig. 1(b), the system is not safe as it is unsafe at $t = 2$. We say that the blue and green agents violate safety.

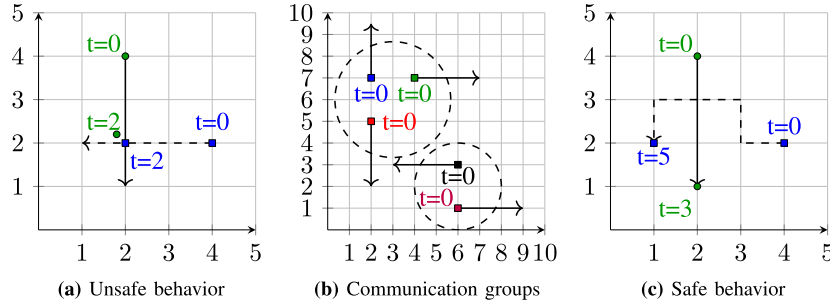


Fig. 1. (a) In Scenario 1, there are two agents (blue and green). Their intended trajectories are marked by solid lines. At time $t = 2$, the blue and green agents occupy the same cell; hence $\varphi(2) = \perp$. However, the system is still safe at times $t = 0$ and $t = 1$. (b) In Scenario 2, the communication constant d is 2. The agents in the same group have been encircled. The black and purple agents are in a group and blue, green, and the red agents are in a different group. (c) In Scenario 4, the modified trajectory for the blue agent as a consequence of the enforcer $S[\text{blue}, 0]$ on the blue agent at $t = 0$ is shown.

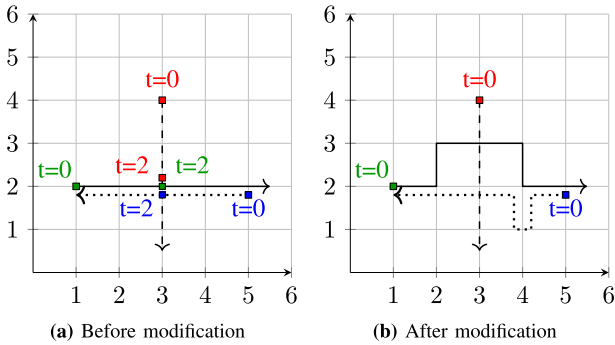


Fig. 2. In Scenario 6, the system is safe after the modification of the trajectories by the enforcers. (a) Blue, green, and red agents violate safety at time 2 in $(3,2)$. (b) Trajectories of the blue and green agents have been modified by the respective enforcers. However, the red agent does not deviate.

Scenario 5: In Fig. 1(b), the blue and green agents occupy the same location at time $t = 2$. However, in Fig. 1(c), the blue agent's trajectory has been modified by the enforcer onboard. As a result, they never occupy the same position at the same time. Priority of the blue agent is 1 and the priority of the green agent is 2. $S_1[\text{blue}, 0]((\text{blue}, 1, \text{lll})(\text{green}, 2, \text{ddd})) = (\text{blue}, 1, \text{ltlld})(\text{green}, 2, \text{ddd})$.

Scenario 6: In Fig. 2 (a), at $t = 0$, blue agent has priority 1, green agent has priority 2, and red agent priority 3. The communication constant $d = 2$. All the three agents occupy the same location $(3,2)$ at time 2. The enforcer on the blue agent modifies its trajectory first and this modification is relayed to the green and the red agents. The enforcer on the green agent then modifies its trajectory. There is no change in the trajectory of the red agent as now there is no safety violation. The modified trajectories are shown in Fig. 2(b).

IV. ONLINE SYNTHESIS

Informally, the enforcer $S[u, t]$ onboard agent u can only modify the trajectory of agent u . Every enforcer has access to a *pathfinder* that modifies the corresponding trajectory. If a potential safety violation is detected, the enforcer on the agent with the lowest priority calls the pathfinder first. The order $\prec_t$

determines the next agent potentially required to modify its trajectory. The pathfinder resolves conflicts, if any, within the group. If a new agent comes into the group, the pathfinder is called by the enforcer on the lowest priority agent. The trajectory of an agent is not modified if it is not involved in a safety violation. An *ordering mechanism* maintains the order $\prec_t$ among the agents. When an agent reaches its final state, then its intended trajectory is updated and the ordering mechanism also updates the priorities.

Pathfinder: Informally, the pathfinder returns a new path whenever called. It constructs a graph and searches for a path in the graph from a vertex corresponding to the current location to a vertex in a target set corresponding to the agent's final state. This graph does not have any outgoing edges from vertices that correspond to unsafe configurations. After a single call to the pathfinder, the maximum length of the modified trajectory is at most $\ell + k$, where $\ell \leq k < d$ is some constant. But such a path may not always exist. In this case, the pathfinder returns a path that at the minimum it keeps the system safe.

Assumption 1: The graph G is 2-edge connected and there is self-loop on every vertex in G .

The self-loop at every vertex in G ensures that the agents can loiter. For any agent $u \in \mathcal{U}$, its trajectory p_u is $p_u = (v_u, w_u)$. The final state of agent u is v_u^f and $\text{priority}(u, t)$ is its priority at time t .

Scenario 7: Fig. 3 depicts the graph G_{blue}^0 constructed by the pathfinder on the blue agent for the example in Fig. 2(a). The initial position is $v_{\text{init}} = ((4, 2), 0)$ and the target set $F = \{((1, 2), 4), ((1, 2), 5), ((1, 2), 6))\}$ is marked red. The nodes occupied by some other higher priority agent at the time are marked by black circles. These black nodes do not have any outgoing edges. The pathfinder returns a path from v_{init} to some vertex in F . The positions of the green agent are unmarked as it has a lower priority.

Scenario 8: For the safety function defined in Scenario 3, the pathfinder constructs the graph $G_u^t = (V', E')$ where $V' = V \times [t, t + d(v_u, v_u^f) + k]$. There is an edge with label e between (v, t) and $(v', t' + 1)$, if in G there is an edge (v, v') with label e and $\mathcal{X}(v, t') = \emptyset$, i.e., there are no higher priority agents occupying the same state. The target set F is $\{(v_u^f, t') | t + d(v_u, v_u^f) \leq t' \leq t + d(v_u, v_u^f) + k\}$ and the initial state is $v_{\text{init}} = (v_u, t)$.

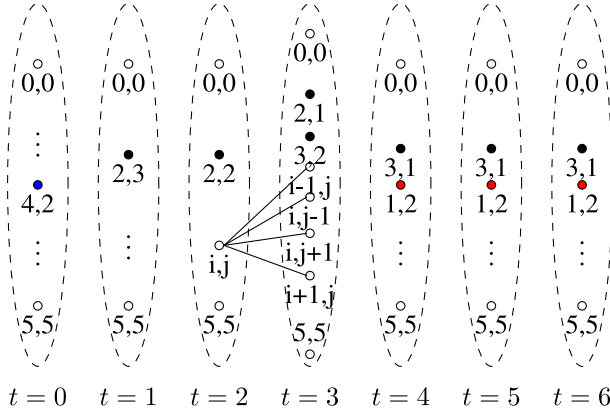


Fig. 3. Graph G_{blue}^0 (see Scenario 7) constructed by the pathfinder for the blue agent is shown. The set of vertices is $\{0, \dots, 5\} \times \{0, \dots, 5\} \times \{0, \dots, 6\}$. There is an edge between $((i, j), t_1)$ and $((i', j'), t_2)$ if and only if $|i' - i| + |j' - j| = 1$ and $t_2 = t_1 + 1$ and there is no out-edge from a black vertex (a black vertex corresponds to it being occupied by some agent at the corresponding time). v_{init} is blue and the vertices in F are red. The positions of the green agent are unmarked as it has a lower priority than the blue agent.

Occupancy Graph: The occupancy graph O_u^t is similar to the pathfinder graph; however, none of the edges are removed from the occupied states. The occupied states are labeled with the corresponding agents. Formally, the occupancy graph $O_u^t = (V', E')$ where $V' = V \times [t, t + d(v_u, v_u^f) + k] \setminus \{(v, t') | v \text{ is occupied by highest priority agent at } t'\}$. There is an edge with label e between (v, t) and $(v', t' + 1)$, if in G there is an edge (v, v') with label e . If $u' \in \mathcal{X}(v, t)$ and $\text{priority}(u) \geq \text{priority}(u')$, then $u' \in \mathcal{X}((v, t))$.

General Pathfinder Graph: Next, we present the pathfinder construction for any local safety property φ for agent u . For this, we need the safety function $\bar{\varphi}_u(t)$ defined as

$$\bar{\varphi}_u(t) \stackrel{\text{def}}{=} \bigwedge_{\text{priority}(u) \leq \text{priority}(i)} \varphi_i(t). \quad (10)$$

The safety function $\bar{\varphi}_u(t)$ ensures that all the agents with priorities higher than u are safe. $G_u^t = (V', E')$ is the graph whose nodes V' are $V' = P \times [t, t + d(v_u, v_u^f) + k]$. Recall $P = V \times 2^U$. There is an edge between (v, t_1) to (v', t_2) if i) $\varphi_u(v) = \top$, ii) (v, v') is an edge in G , iii) $t_2 = t_1 + 1$, iv) $\bar{\varphi}_u(t) = \top$, and v) all the other higher priority agents are following their trajectories, i.e.,

$$u_i \in \mathcal{X} \left(\hat{\delta}(u_i^t, \prod_i W[0 : t_1]), t_1 \right) \quad (11)$$

$$\text{and } u_i \in \mathcal{X} \left((\hat{\delta}(u_i^t, \prod_i W[0 : t_2]), t_2) \right). \quad (12)$$

The initial vertex v_{init} is (a, t) where $a = v_u \times \mathcal{X}(v_u, t)$ and the set of target vertices F is $F = \{(v, j) | v \in P' \text{ and } t + d(v_u, v_u^f) \leq j \leq t + d(v_u, v_u^f) + k\}$, where $P' \subseteq P$ is a subset of all vertex labeling such that agent u has reached its final state. Next, we describe the working of the pathfinder. The pathfinder constructs the graph G_u^t , initializes v_{init} , a set F of target vertices. The pathfinder then returns a path from v_{init} to some state in F if it exists, else it returns a random path of length 1 that only ensures safety. If no such path exists, the agent finds a shortest path in the

Algorithm 1: Pathfinder on Agent u .

Result: Safe path for u
Initialize G_u^t and O_u^t ;
Initialize v_{init} and F ;
 P = path in G_u^t from v_{init} to F ;
if P exists **then**
 return p ;
end
else
 P = shortest path to an unoccupied vertex in O_u^t ;
 forall vertex $v_i \in P$ **do**
 if $\mathcal{X}(v_i, t) = a$ **then**
 $w_a = (v_i, v_{i+1})$;
 end
 end
end

occupancy graph to a vertex that ensures safety. This path may have other agents. All the other agents are forced to move one step along this path. If $p = v_1 v_2 \dots v_i v_{i+1}$ is the path from the occupancy graph, and agent m is at v_i , then the trajectory of m is replaced by (v_i, v_{i+1}) . In short, if some lower priority agent cannot plan around the higher agent, then it might disturb all the agents other than the highest priority agent to ensure safety. However, the highest priority agent's path cannot be changed. In the worst case, the pathfinder ensures that the agent with the highest priority can progress without any modifications. In essence lower priority agents progress, if they can plan around the highest priority agent.

Ordering Mechanism: The priorities of the agents cannot remain static with time. Otherwise, some agent might be forced to change its trajectory infinitely often. In the sequel, we present the ordering mechanism.

Overview of Ordering Mechanism for Two Agents:: Consider a system with two agents a and b that have communicated, i.e., observed each other's trajectories. Agent a maintains a flag c_a^b and agent b maintains a flag c_b^a . If agent a has reached its final state after communicating with agent b , then the flag c_a^b is set to 1. Suppose agent b is yet to reach its final state and there is a safety violation after a has completed its goal; then, in order to ensure freedom from locks, agent a is forced to modify its trajectory. When agent b reaches its final state, both agents have uniformly completed their goals and the flags are reset to 0. The earlier procedure is equivalent to the standard binary semaphores algorithm to achieve process synchronization [21].

Scenario 9: In Fig. 4, the agents are following the modified trajectories in Fig. 1(c). The priority of the agent changes once the agent reaches its final state. More precisely, the agent gets the lowest priority once it reaches its final state.

Extension to Arbitrary Number of Agents:

We extend the procedure outlined earlier to multiple agents. Each pair of agents u_i and u_j maintain two Booleans between them (each of them is analogous to a binary semaphore) that are used to measure relative progress. Formally, $\bar{C}_u = (c_u^1, c_u^2, \dots, c_u^{|U|})$ is a vector of Boolean flags for maintaining progress of u with respect to the other agents and $\bar{B}_u$ is a set

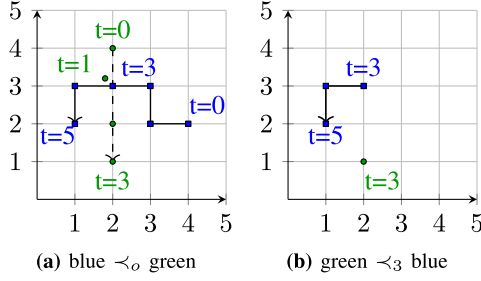


Fig. 4. In Scenario 9, the blue and green agents are following their modified trajectories from Scenario 3. Initially, the blue agent has a lower priority; hence, it is forced to modify its path. When the green agent reaches its final state at $t = 3$, the green agent is assigned a lower priority. The blue agent's priority is higher than the green agent's. Again at $t = 5$, the priorities change since the blue agent has reached its final state.

maintained by u for tracking the agents it has communicated with during the current final state. Initially, $B_u = \emptyset$ and $\overline{C}_u = \overline{0}$. The flag c_u^v on agent u records the progress of u with respect to v . If c_u^v is 1, agent u has recorded that it has finished a goal ($\text{goal}_{u,t} = \top$) after communicating with v ($v \in B_u$). Whenever the corresponding progress measures c_u^v and c_v^u are equal and the agents are in the same communication group, the Boolean flags are reset to 0. The exact algorithm is presented in Fig. 5(b).

Let $c_u^v(t)$ denote the value of the flag c_u^v at time t . We use $a \prec^t b$ to denote that $c_a^b(t) = 1$, $c_b^a(t) = 0$ and $a =^t b$ to denote $c_a^b(t) = c_b^a(t)$. Observe that for any pair of agents a and b either $a =^t b$ or $a \prec^t b$ or $b \prec^t a$.

Proposition 1: If $a \prec^t b$ and $b \prec^t c$, then no agent d exists such that $c \prec^t d$ and $d \prec^t a$.

Proof by Contradiction: For any agent u , let $\text{comp}(u)$ denote the earliest time t' such that $\text{goal}_{u,t'} = \top$. If $a \prec^t b$, then $c_a^b = 1$ and $c_b^a = 0$, i.e., agent a has reached its final state, but agent b has not. Similarly, $b \prec^t c$ implies that agent b has reached its final state, but agent c has not. Therefore

$$\text{comp}(a) < \text{comp}(b) < \text{comp}(c). \quad (13)$$

Now, suppose that there exists an agent d such that $c \prec^t d$ and $d \prec^t a$, then by the same argument, the order of last completed goals among a , c , and d is

$$\text{comp}(c) < \text{comp}(d) < \text{comp}(a). \quad (14)$$

Equation (13) contradicts (14). Therefore, there cannot exist agent d such that $c \prec^t d$ and $d \prec^t a$. ■

Corollary 1: $\prec^t$ is a partial order.

Proposition 2: There exists a total order $\prec_t$ that respects $\prec^t$.

Proof: Define $\prec_t$ as

$$\begin{aligned} a \prec_t b & \text{ if i) } a \prec^t b \text{ or ii) } a =^t b \text{ and } a \prec_0 b \\ b \prec_t a & \text{ otherwise} \end{aligned} \quad (15)$$

where a and b are some agents. $\prec_t$ as defined is a total order. ■

Henceforth, we use $\prec_0$ to generate $\prec_t$ that respects $\prec^t$.

Decentralized Enforcement

So far, we have described the components of the enforcers onboard an agent. In traditional shield synthesis, S_1 is a function

that is fully constructed and used as the shield [2]. In contrast, here S_1 is a partial function that is constructed when required. Fig. 5(a) presents the algorithm to determine the calls to the pathfinder and Fig. 5(b) presents the ordering mechanism to update the priorities by modifying the corresponding flags. Scenario 10 demonstrates the use of the flags to ensure a total order among the agents.

For some agent u_i , if w'_{u_i} is the path returned by the pathfinder, then

$$S_1[u, t](O, V, W) = (O, V, \mathcal{R}_i(W, (v_{u_i}, w'_{u_i}))).$$

That is, the path for the agent u_i has been replaced with w'_{u_i} , with the paths of the other agents unaffected and their priorities unchanged. If the pathfinder is never called, then the path does not get modified, i.e., $S_1[u_i, t](O, V, W) = (O, V, W)$.

The following lemma is a direct consequence of the construction of the pathfinder graph G_u^t and a path between v_{init} and F .

Lemma 1: For all $t' \in [t, t + \ell]$ and $u \in \mathcal{U}$, if agent u moves along the trajectory returned by $S_1[u, t']$, then $\varphi(t') = \top$.

Next, we prove that the agent with the highest priority is able to progress without any deviation.

Lemma 2: If agent u has the highest priority according to $\prec_t$, then it will reach its final state without any modifications. Moreover, if a ℓ -stabilizing centralized shield can ensure safety, then the enforcers also can ensure safety.

Proof: If u has the highest priority by $\prec_t$, then for all $v \in \mathcal{U}$, it is either the case that $c_u^v = 0$ and $c_v^u = 1$ (or) $c_v^u = c_u^v$ and $v <_0 u$. In either case, v finds a new path if a safety violation is detected. Since a centralized shield can ensure safety, it implies that there is at least one safe position for v in the occupancy graph O_v^t . By Assumption 1, G is 2-edge connected. Therefore, there is a safe vertex such that the path length is at most ℓ . By the pathfinder algorithm, the trajectory of v is modified. Similarly, all other agents modify their trajectories in the case of a safety violation. ■

Corollary 2: In the worst case, the distance between u and its final state maybe $|\mathcal{U}|(\ell)$.

We now prove that the other agents are also guaranteed to make progress. The following theorem bounds the maximum deviation from the original trajectory.

Lemma 3: Enforcer on agent u may cause a deviation from the intended trajectory for at most $|\mathcal{U}|^2(\ell)$ steps before the final state is reached.

Proof: In the worst case, any agent u may be forced to use the occupancy graph during every call to the pathfinder, before agent u has the highest priority according to $\prec_t$. In worst case, agent u might be the lowest priority vertex to start with. Hence, agent u may require $|\mathcal{U}| - 1(|\mathcal{U}|\ell)$ steps to get the highest priority. At this stage by Corollary 2, agent u requires $|\mathcal{U}|(\ell)$ steps to reach its goal. ■

The next theorem establishes that the enforcers we synthesize satisfy the properties stated in Problem 1.

Theorem 1: The set $\{S[u, t] | u \in \mathcal{U}\}$ of enforcers are i) correct, ii) deviate minimally, iii) bounded, and iv) complete.

Proof: As a consequence of Lemma 3, the maximum number of steps that any agent needs to reach its final state is bounded by

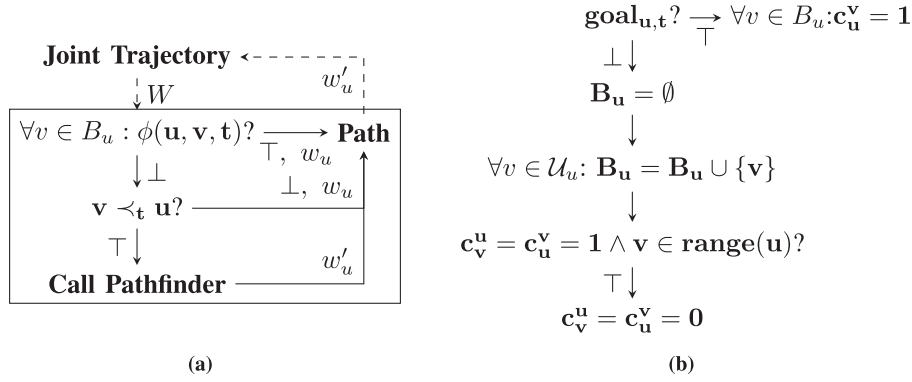


Fig. 5. (a) Algorithm for the enforcer $S(u, t)$ to decide if the pathfinder should be called at time t . (b) Algorithm to maintain the priorities.

($|\mathcal{U}|^2\ell$). Therefore, the synthesized enforcers are all $(\ell, |\mathcal{U}|^2\ell)$ -enforcers and by Definition 3 are *bounded*. In the algorithm in Fig. 5(a), if no safety violation is detected, then the pathfinder is never called by the enforcer. Therefore, by Definition 2, the enforcers also *deviate minimally*. By Definition 4 (completeness), we only require that if a centralized shield can ensure safety, then the enforcers can ensure safety. Lemma 2 ensures the enforcer can ensure safety, if a ℓ -stabilizing centralized shield can ensure safety, i.e., it is *complete*. Finally, Lemma 1 establishes that the enforcers are *correct*. ■

The main complexity result of the article, where we bound the worst case synthesis time, is formalized in the theorem ahead.

Theorem 2: Given fixed look-ahead ℓ and maximum deviation length k , the enforcer on an agent takes $\mathcal{O}(|\mathcal{U}|^2)$ time to modify the corresponding agent's trajectory.

Proof: The pathfinder constructs a graph for agent u of size at most $(k + \ell)|\mathcal{U}|$. If k and ℓ are fixed, then the size of the graph constructed by the pathfinder is LINEAR($|\mathcal{U}|$). Moreover, the number of edges in this graph is also at most $(k + \ell)|\mathcal{U}|$. The time complexity of solving a search in this graph is $\mathcal{O}((k + \ell)|\mathcal{U}|)$. In the worst case, all agents are in the same communication group and the lowest priority agent may have to modify its trajectory at most $|\mathcal{U}|$ times. ■

Remark 1: Theorem 2 implies that the complexity of finding a safe trajectory depends only on the number $|\mathcal{U}|$ of agents, maximum deviation length k , and look-ahead length ℓ and does not depend on the size $|V| + |E|$ of the environment G .

V. EXTENSION TO TRAJECTORIES OF ARBITRARY LENGTH

In problem 1, we assume that the length of any trajectory is bound by some constant ℓ . In this section, we define the problem for trajectories of arbitrary length and propose a solution. We show how to use the ordering mechanism to ensure that the enforcers can ensure safe behavior even when the trajectories are of arbitrary length.

Problem 2: Given a set $\mathcal{U} = \{u_1, \dots, u_n\}$ of agents and a set $\{(v_u, w_u) | u \in \mathcal{U}\}$ of their trajectories, construct a set $\{S[u, t] | u \in \mathcal{U} \text{ and } t \in \mathbb{T}\}$ of enforcers such that these enforcers are correct, cause minimum deviation and bounded.

Luckily, we do not have to change the entire synthesis procedure to solve this problem. We artificially restrict the input to

Algorithm 2: Enforcer on Agent u .

Result: Trajectory of length at most length $\ell + k$
 $i = 0$;
Initialize w_u and v_u ;
 $v_u^f = \hat{\delta}(v_u, w_u(0)[0 : \ell])$;
 $w_u(0) = w_u[0 : \ell]$;
while *True* **do**
 Update Communication Groups ;
 Increment Time ;
 if $\text{goal}_{u,t} = \top$ **then**
 $i++$;
 $v' = v_u^f$;
 $w_u(t) = w_u[i\ell : (i+1)\ell]$;
 $v_u^f = \hat{\delta}(v_u, w_u[i\ell : (i+1)\ell - 1])$;
 $v_u = v'$;
 end
 forall v in $\mathcal{U}_u$ **do**
 if $(u, v \text{ violate safety}) \wedge (v =_t u)$ **then**
 if $v \prec_0 u$ **then**
 Call the pathfinder on u and find a new path ;
 end
 end
 else if $(u, v \text{ violate safety}) \wedge (v \prec_t u)$ **then**
 Call the pathfinder on u and find a new path ;
 $B_u = \emptyset$;
 end
 end
 $t++$;
end

the enforcers. Let $w_u(0)$ be the trajectory (possibly infinite) for agent u and v_u be the start state of the agent. We divide w_u into blocks of length ℓ , $w_u(0) = w_u[0 : \ell - 1] \cdot w_u[\ell : 2\ell - 1] \dots$. The enforcer $S[u, 0]$ uses $w_u(0)$ as its trajectory; once it reaches the final state $\hat{\delta}(v_u, w_u(0))$, it uses $((\hat{\delta}(v_u, w_u(0)), w_u[\ell : 2\ell - 1])$ as the new trajectory for the enforcer synthesis. Algorithm 2 describes this procedure.

In the algorithm, whenever the agent reaches its final state, its trajectory is updated with the next ℓ moves and its flags are also suitably reset. The correctness of this procedure is a direct consequence of the main theorem.

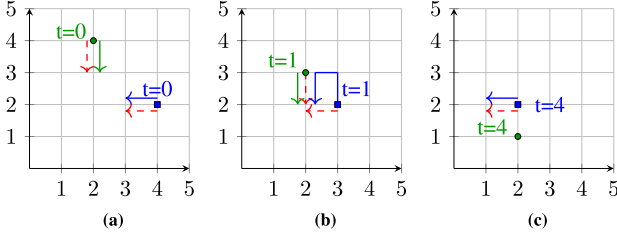


Fig. 6. In Scenario 10, the trajectories of agents have been modified by a decentralized enforcer with $\ell = d = 1$. Dashed arrows correspond to the intended trajectories and solid arrows correspond to the modified trajectories. The agents detect a possible collision at $t = 1$ and the blue agent b is forced to deviate by the enforcer, whereas the green agent g does not deviate. (a) $t = 0$. (b) $t = 1$. (c) $t = 4$.

VI. EXPERIMENTAL EVALUATION

We evaluate the performance of the runtime decentralized enforcer synthesis framework in the context of collision avoidance for multiagent systems. Specifically, we use the collision-avoidance safety function defined in Scenario 3 and the pathfinder construction described in Scenario 7. The implementation of the system for ensuring safety from collisions uses the general pseudocode presented in Algorithm 2.

Comparison With Centralized Shields: We compare the modified trajectories of two agents equipped with decentralized enforcers that are synthesized at runtime with two other agents whose behaviors are modified by a centralized shield synthesized at design-time using the algorithm from [4] in a 5×5 grid world. This algorithm runs in time that is exponential in the number $|\mathcal{U}|$ of agents [4]. In general, the optimization of the plans of cooperative robots is PSPACE-hard [22]. The intended trajectories of the agents in both scenarios are the same.

Decentralized enforcers can incorporate look-ahead $\ell \geq 1$. In the case with $\ell = 1$, the decentralized enforcers behave precisely the same as the centralized enforcer as they can only detect collisions in the next step and we illustrate such a scenario ahead.

Scenario 10: In Fig. 6, at time $t = 0$, the two agents are in different communication groups and $b \prec_0 g$. At time $t = 1$, the two agents are in the same communication group. Therefore, $B_b = \{g\}$, $B_g = \{b\}$ and the flags $c_b^g = c_g^b = 0$. At time $t = 4$, the green agent has reached the goal, i.e., $c_g^b = 1$. Thus, $g \prec_4 b$. At time $t = 5$, the blue agent reaches its goal, i.e., $c_b^g = 1$. Therefore, $b \prec_5 g$. Finally, both c_b^g and c_g^b are reset to 0.

In the case with $\ell = 2$, the enforcers induce a different behavior. At $t = 0$, only the collision is detected, but the final state for the blue agent is (2,2) instead of (1,2) as in the previous case. The intended trajectory is updated when the agents have reached their current goals (in this case, this update happens at (2,2) for both the agents). The effect of the enforcer is shown in Fig. 7. In the case with look ahead $\ell = 3$, i.e., with further look ahead, we recover the solution presented in Fig. 4.

As shown in these scenarios, the look-ahead parameter ℓ impacts the modified behavior. The agent has an increased ability to prevent future collisions with a larger value of ℓ . This enhanced ability to prevent collisions comes at the cost of the synthesis time as the size of the graph constructed by the pathfinder increases. But, it does not affect the maximum length of the deviation.

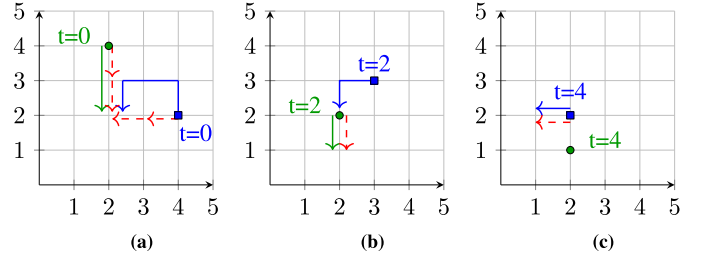


Fig. 7. Trajectories of the blue and green agents have been modified by the respective enforcers acting on them to ensure no collisions. The first goal for the two agents is (2,2), as their look-ahead $\ell' = 2$. Once they reach (2,2), their intended trajectories are updated, which is shown in (b) and (c). (a) $t = 0$. (b) $t = 2$. (c) $t = 4$.

Scalability: We built a multiagent system where the agents are equipped with the decentralized enforcer framework for collision avoidance in Python. The distributed nature of the system is modeled using shared memory. The size of the grid world, the look-ahead length ℓ , the communication constant d , and the length k of the maximum deviation by one use of the pathfinder are user inputs. The original trajectories for the agents are random. We record the effect of ℓ and k on the synthesis time for modified behavior. The results are obtained on an Intel Core-i7 CPU @ 2.2 GHz with 16 GB of RAM. We set $d = \ell$ in all the experiments. We show the results of these experiments in Table I.

To synthesize the centralized shield, a safety game is solved. We show the size of the game graph (in the order of magnitude) for the different scenarios. The large size explains why the design-time synthesis of centralized shields is infeasible in multiagent settings. For comparison, we also show the exact size of the graph constructed by the pathfinder for each scenario. Finally, we record the best case and the worst case synthesis times in the decentralized setting. Observe that the synthesis times are the same if ℓ, k , and $|\mathcal{U}|$ are the same. Table I shows that the synthesis time does not depend on the number of states in the environment. This observation is consistent with the earlier analysis. The worst case is when all the agents interfere with one another and they have global information. In this scenario, only the agent with the highest priority can progress without modifications. Every other agent has to wait for the agents with a lower priority to fix their trajectories. Table I shows that the synthesis time is in the order of a few seconds.

ROS/Gazebo Simulation: In this section, we demonstrate the implementation of the enforcers in a high-fidelity simulation discrete environment, as shown in Fig. 8. The transition of each vehicle from one cell to another cell is synchronized and timed, i.e., when a vehicle is assigned its target cell; it transits to the target cell and waits until all the other vehicles reach their targets before moving toward its next target. We *do not* use any in-built obstacle avoidance library for low-level collision avoidance.

We tested two different scenarios in this environment. In the first scenario, we used a new safety function to prevent collision with obstacles (buildings) and ensure that the Manhattan distance between any two agents is at least 2 units. We randomly generated trajectories for the agents. We consider a scenario with eight agents in this environment. The even agents have the same

TABLE I

COMPARISON OF STATE SPACE SIZES BETWEEN CENTRALIZED AND DECENTRALIZED ONLINE ENFORCEMENT APPROACHES WITH REPORTED SYNTHESIS TIMES FOR THE DECENTRALIZED APPROACH

$ \mathcal{U} $	States	ℓ	k	Centralized game graph		Decentralized pathfinder graph		$ \mathcal{L}_I $	$ \mathcal{L}_O $	Decentralized synthesis time (best) (worst)	
				$ V $	$ E $	$ V $	$ E $				
3	3^2	3	3	10^8	10^{12}	18	18	10^2	10^3	0.089	0.267
3	3^2	5	5	10^{11}	10^{18}	30	30	10^3	10^7	0.092	0.276
3	3^2	10	5	10^{20}	10^{30}	45	45	10^7	10^{10}	0.093	0.279
3	5^2	5	5	10^{13}	10^{19}	30	30	10^5	10^7	0.092	0.276
3	10^2	5	5	10^{15}	10^{21}	30	30	10^6	10^7	0.092	0.276
3	50^2	5	5	10^{19}	10^{25}	30	30	10^7	10^7	0.092	0.276
5	3^2	5	5	10^{19}	10^{25}	50	50	10^3	10^7	0.2	1
5	5^2	5	5	10^{22}	10^{28}	50	50	10^5	10^7	0.2	1
5	10^2	5	5	10^{25}	10^{31}	50	50	10^6	10^7	0.2	1
20	50^2	3	3	10^{104}	10^{107}	120	120	10^6	10^6	0.41	8.2
20	50^2	5	5	10^{128}	10^{134}	200	200	10^7	10^7	0.42	8.4
20	50^2	10	5	10^{188}	10^{197}	300	300	10^{10}	10^{10}	1.29	25.8
30	50^2	10	5	10^{282}	10^{291}	450	450	10^{10}	10^{10}	1.62	48.6
40	50^2	10	5	—	—	600	600	10^{10}	10^{10}	1.64	65.6
50	50^2	10	5	—	—	750	750	10^{10}	10^{10}	1.67	83.5
60	50^2	10	5	—	—	900	900	10^{10}	10^{10}	1.7	102

As the enforcers are only synthesized as needed for the relevant agents, we report both the worst case and best case total synthesis times (sec) for all relevant enforcers for every detected collision. In case of the number of vertices and edges in the centralized approach, the order of magnitude are shown. $\mathcal{L}_I$ and $\mathcal{L}_O$ are the input alphabet and the output alphabet, respectively.

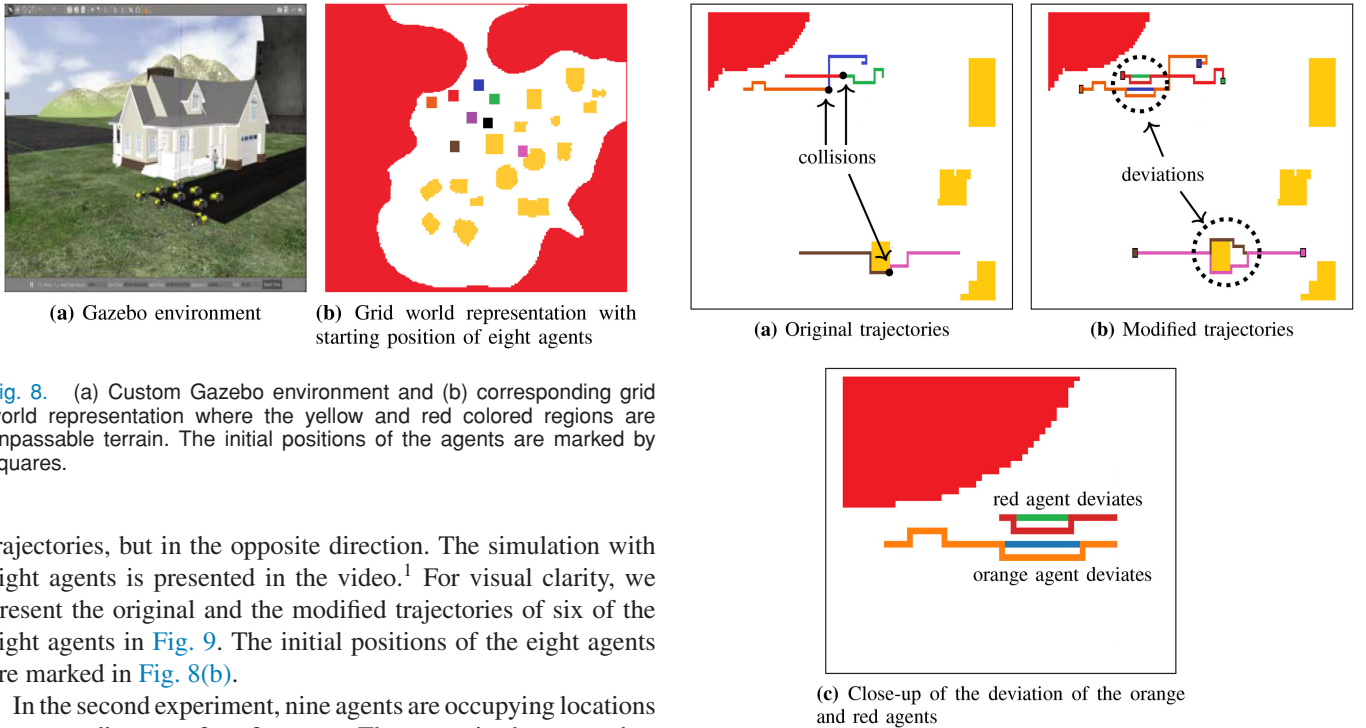


Fig. 8. (a) Custom Gazebo environment and (b) corresponding grid world representation where the yellow and red colored regions are unpassable terrain. The initial positions of the agents are marked by squares.

trajectories, but in the opposite direction. The simulation with eight agents is presented in the video.¹ For visual clarity, we present the original and the modified trajectories of six of the eight agents in Fig. 9. The initial positions of the eight agents are marked in Fig. 8(b).

In the second experiment, nine agents are occupying locations corresponding to a 3×3 square. The agent in the center has to escape the confinement. Furthermore, no trajectories for the other agents are given. In this scenario, the central agent chooses its trajectory when it gets the highest priority. The agents along the path of this agent have to make way. During this process,

¹The video can be found at <https://tinyurl.com/yhddpm6>.

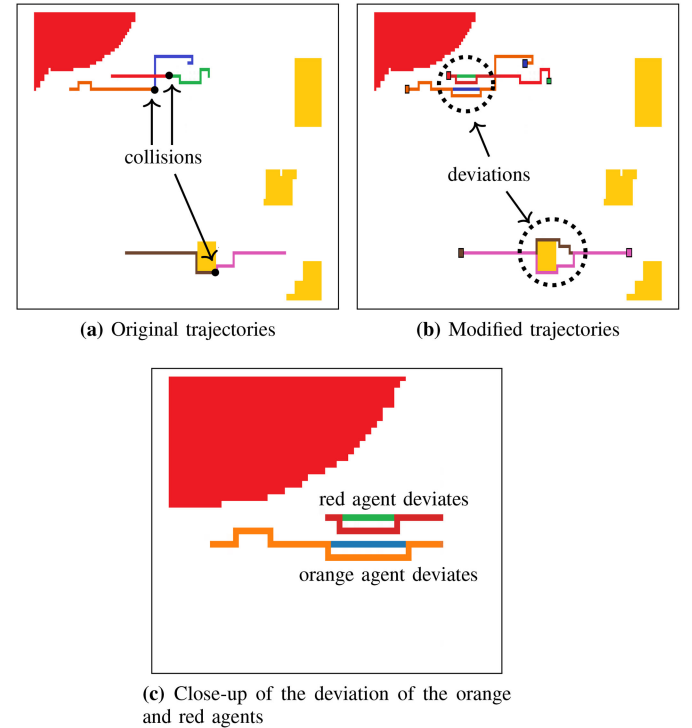


Fig. 9. (a) Original trajectories of the orange, brown, and red agents are in conflict with the trajectories of the blue, pink, and green agents, respectively. The collisions are marked with black dots. (b) Initial positions of the agents are marked by squares. The modified trajectories ensure that there is no collision between the agents and that they are at least two units apart. The deviations are marked by the dashed black circles. (c) We show a close-up of the deviation of the orange and red agents from their original trajectories.

sometimes a set of agents may actually have to make way for one another. This scenario can be seen in the video.² We also provide the data for the trajectories and obstacles can be obtained.³

VII. CONCLUSION

We presented an online synthesis approach for runtime enforcers that guarantee local safety in multiagent systems. The approach is decentralized and safe trajectories are synthesized on-board each agent as necessary. The algorithm we presented does not require global information on the states of the other agents in the system. It only requires the information about the agents in the same communication group. With minor assumptions, we proved the correctness of the approach in enforcing safety and also that all the agents progress toward their original plan by bounding the maximum deviation and proving the absence of deadlocks. Finally, we showed that if an ℓ -stabilizing centralized shield can guarantee correctness, then $(\ell, |\mathcal{U}|^2\ell)$ -enforcers can also guarantee correctness. We further prove that this synthesis scales efficiently with the number of agents.

In this article, we have assumed perfect and instantaneous communication between agents, which is not usually achievable in the real world. For future work, we aim to investigate the problem under the assumption of intermittent communication loss between agents and studying the effects on safety guarantees. Additionally, we plan to consider the enforcement of general safety properties that do not require the *local* restriction. Furthermore, we plan to extend the technique to deal with *nonstatic* environments, i.e., environments where the effect of an action depends on the time when it is performed.

REFERENCES

- [1] Y. Falcone, J. Fernandez, and L. Mounier, "What can you verify and enforce at runtime?" *Int. J. Softw. Tools Technol. Transfer*, vol. 14, no. 3, pp. 349–382, 2012.
- [2] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang, "Shield synthesis," in *Proc. Int. Conf. Tools Algorithms Construction Anal. Syst.*, 2015, pp. 533–548.
- [3] B. Könighofer *et al.*, "Shield synthesis," *Formal Methods Syst. Des.*, vol. 51, no. 2, pp. 332–361, 2017.
- [4] S. Bharadwaj, R. Bloem, R. Dimitrova, B. Könighofer, and U. Topcu, "Synthesis of minimum-cost shields for distributed systems," in *Proc. Annu. Amer. Control Conf.*, Jul. 2019, pp. 10–12.
- [5] S. Bharadwaj, S. Carr, N. Neogi, H. Poonawala, A. B. Chueca, and U. Topcu, "Traffic management for urban air mobility," in *NASA Formal Methods*, J. M. Badger and K. Y. Rozier, Eds. Cham, Switzerland: Springer, 2019, pp. 71–87.
- [6] S. Schewe, "Synthesis of distributed systems," Ph.D. Dissertation, Saarland Univ., Saarbrücken, Germany, 2008.
- [7] D. Silver, "Cooperative pathfinding," in *Proc. 1st AAAI Conf. Artif. Intell. Interact. Digit. Entertainment*, Jun. 2005, pp. 117–122.
- [8] J. E. Hopcroft, J. T. Schwartz, and M. Sharir, "On the complexity of motion planning for multiple independent objects; PSPACE-hardness of the 'Warehouseman's problem,'" *Int. J. Robot. Res.*, vol. 3, no. 4, pp. 76–88, Dec. 1984.
- [9] T. Standley, "Finding optimal solutions to cooperative pathfinding problems," in *Proc. 24th AAAI Conf. Artif. Intell.*, Jul. 2010, pp. 173–178.
- [10] T. Standley and R. Korf, "Complete algorithms for cooperative pathfinding problems," in *Proc. 22nd Int. Joint Conf. Artif. Intell.*, 2011, pp. 668–673.
- [11] Y. Zhang, K. Kim, and G. Fainekos, "DisCoF: Cooperative pathfinding in distributed systems with limited sensing and communication range," in *Proc. Distrib. Auton. Robot. Syst.*, 2016, pp. 325–340.
- [12] P. Zhu, C. Liu, and S. Ferrari, "Adaptive online distributed optimal control of very-large-scale robotic systems," 2020, *arXiv: 2003.01891*.
- [13] P. Zhu, S. Ferrari, J. Morelli, R. Linares, and B. Doerr, "Scalable gas sensing, mapping, and path planning via decentralized Hilbert maps," *Sensors*, vol. 19, 2019, Art. no. 1524.
- [14] K. Rudd, G. Foderaro, P. Zhu, and S. Ferrari, "A generalized reduced gradient method for the optimal control of very-large-scale robotic systems," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1226–1232, Oct. 2017.
- [15] S. Al-Abri, T. X. Lin, M. Tao, and F. Zhang, "A derivative-free optimization method with application to functions with exploding and vanishing gradients," *IEEE Control Syst. Lett.*, vol. 5, no. 2, pp. 587–592, Apr. 2021.
- [16] R. Bloem, R. Könighofer, and M. Seidl, "SAT-based synthesis methods for safety specs," in *Proc. Int. Conf. Verification, Model Checking, Abstract Interpretation*, 2014, pp. 1–20.
- [17] Y. Falcone, L. Mounier, J.-C. Fernandez, and J.-L. Richier, "Runtime enforcement monitors: Composition, synthesis, and enforcement abilities," *Formal Methods Syst. Des.*, vol. 38, no. 3, pp. 223–262, Jun. 2011.
- [18] S. Pinisetty, P. S. Roop, S. Smyth, S. Tripakis, and R. von Hanxleden, "Runtime enforcement of reactive systems using synchronous enforcers," in *Proc. 24th ACM SIGSOFT Int. SPIN Symp. Model Checking Softw.*, 2016, pp. 80–89.
- [19] Y. Falcone, J.-C. Fernandez, and L. Mounier, "Synthesizing enforcement monitors wrt. the safety-progress classification of properties," in *Proc. Inf. Syst. Secur.*, 2008, pp. 41–55.
- [20] D. B. West, *Introduction to Graph Theory*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice-Hall, Sep. 2000.
- [21] A. Silberschatz, G. Gagne, and P. B. Galvin, *Operating System Concepts*. Hoboken, NJ, USA: Wiley, 2018.
- [22] E. Xidias, P. Zacharia, and A. Nearchou, "Path planning and scheduling for a fleet of autonomous vehicles," *Robotica*, vol. 1, pp. 1–17, 2015.
- [23] J. R. Kok, M. T. Spaan, and N. Vlassis, "Multi-robot decision making using coordination graphs," in *Proc. 11th Int. Conf. Adv. Robot.*, vol. 3, 2003, pp. 1124–1129.
- [24] T. Wongpiromsarn, U. Topcu, and R. Murray, "Formal synthesis of embedded control software: Application to vehicle management systems," in *Proc. AIAA Infotech@Aerospace*, 2011, Art. no. 1506.
- [25] P. Sujit, A. Sinha, and D. Ghose, "Multi-UAV task allocation using team theory," in *Proc. 44th IEEE Conf. Decis. Eur. Control*, 2005, pp. 1497–1502.
- [26] J. H. Ong, A. Sanchez, and J. Williams, "Multi-UAV system for inventory automation," in *Proc. 1st Annu. RFID Eurasia*, 2007, pp. 1–6.
- [27] S. Rasmussen, D. Kingston, and L. Humphrey, "A brief introduction to unmanned systems autonomy services (UxAS)," in *Proc. IEEE Int. Conf. Unmanned Airc. Syst.*, 2018, pp. 257–268.
- [28] C. Guestrin, D. Koller, and R. Parr, "Multiagent planning with factored MDPs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2002, pp. 1523–1530.
- [29] A. Pnueli, "The temporal logic of programs," in *Proc. IEEE 18th Annu. Symp. Found. Comput. Sci.*, Nov. 1977, pp. 46–57.
- [30] E. Bartocci and Y. Falcone, Eds., *Lectures on Runtime Verification—Introductory and Advanced Topics (Lecture Notes in Computer Science)*. Berlin, Germany: Springer, 2018, vol. 10457.
- [31] A. Bauer, M. Leucker, and C. Schallhart, "Runtime verification for LTL and TLTL," *ACM Trans. Softw. Eng. Methodol.*, vol. 20, no. 4, pp. 14:1–14:64, 2011.
- [32] A. Bauer and Y. Falcone, "Decentralised LTL monitoring," *Formal Methods Syst. Des.*, vol. 48, nos. 1/2, pp. 46–93, 2016.



Dhananjay Raju received the B.Tech. degree in computer science and engineering from Amrita University, Coimbatore, India, in 2011, and the M.Sc. degree in computer science from Chennai Mathematical Institute, Chennai, India, in 2014. He is currently working toward the Ph.D. degree in computer science at The University of Texas at Austin, Austin, TX, USA.

His research interests include the use of formal methods, game theory, and logic for the design and verification of autonomous systems.

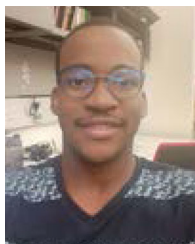
²The video can be found at <https://tinyurl.com/ygwaplcp>.

³Numpy encodings of the trajectories and obstacles can be found at <https://tinyurl.com/y6mkvcdx>



Sudarshanan Bharadwaj (Member, IEEE) received the B.Sc. and B.E. degrees in applied mathematics and aerospace engineering from The University of Sydney, Sydney, NSW, Australia, in 2014, and the M.S. degree in aerospace engineering in 2016 from The University of Texas at Austin, Austin, TX, USA, where he is currently working toward the Ph.D. degree with in aerospace engineering and engineering mechanics.

His research interests include the intersection of formal methods, reinforcement learning, and control with a focus on provable safety guarantees.



Franck Djeumou received the B.S. and M.S. degrees in aerospace engineering from ISAE-SUPAERO, Toulouse, France, in 2018, and the M.S. degree in computer science from École Polytechnique, Palaiseau, France, in 2017. He is currently working toward the Ph.D. degree in electrical and computer engineering at The University of Texas at Austin, Austin, TX, USA.

His current research interests include optimization in formal methods, control theory, and reinforcement learning with provable guarantees.



Ufuk Topcu (Member, IEEE) received the Ph.D. degree mechanical engineering from the University of California Berkeley, Berkeley, CA, USA, in 2008.

He is currently an Associate Professor with The University of Texas at Austin, Austin, TX, USA. He holds the W.A. "Tex" Moncrief, Jr. Professorship in computational engineering and sciences. He held a postdoctoral research position with the California Institute of Technology, Pasadena, CA, USA, until 2012, and was a Research Assistant Professor with the University of Pennsylvania, Philadelphia, PA, USA, until 2015.

His research interests include the theoretical, algorithmic, and computational aspects of design and verification of autonomous systems through novel connections between formal methods, learning theory, and controls.

Dr. Topcu was the recipient of the Antonio Ruberti Young Researcher Prize, the NSF CAREER Award, and the Air Force Young Investigator Award.